Ejercicio guiado: Sistema POS (Point of Sale) para supermercado

Curso Programación II sección 1 — Proyecto integrador

1. Descripción del problema

Un supermercado tiene varias **cajas registradoras**. Cada caja es operada por **un solo cajero a la vez**, pero distintos cajeros se turnan durante el día. Construirás una aplicación de escritorio en Java puro (Swing + JDBC + Oracle XE) que cubra todo el ciclo de trabajo de una caja:

1. Autenticación (login).
2. Apertura de turno con **declaración del efectivo inicial** (fondo para dar cambio).
3. Cobro de mercadería a clientes.
4. **Corte intermedio**: el cajero entrega parte del efectivo para que se resguarde en la bóveda.
5. **Reversión** de un producto ya cobrado, que exige autorización de un usuario con un rol superior.
6. **Corte final** del turno con cuadre de caja.
7. Administración de usuarios, roles y permisos.

Objetivos de aprendizaje

- Modelar un dominio real con clases Java puras (encapsulamiento, enums, composición).
- Escribir SQL y acceder a Oracle XE con JDBC (`PreparedStatement`, `ResultSet`).
- Aplicar seguridad sin frameworks: hash de contraseñas con PBKDF2, permisos por rol y re-autenticación puntual.
- Manejar dinero correctamente con `BigDecimal` y transacciones JDBC (`commit` / `rollback`).
- Construir pantallas con Swing y separar capas: ventana, servicio y DAO.
- Probar reglas de negocio con tu propio ejecutor de pruebas.

Tecnologías permitidas

Componente	Tecnología
Lenguaje	Java 21 (solo la biblioteca estándar)
Interfaz gráfica	Swing (`javax.swing`, incluido en el JDK)
Base de datos	Oracle Database XE 21c
Acceso a datos	JDBC puro (`java.sql`) con el driver `ojdbc11.jar`
Seguridad de contraseñas	PBKDF2 con `javax.crypto` (incluido en el JDK)
Compilación	`javac` desde consola o desde el IDE, sin Maven ni Gradle
Pruebas	Clase propia con `main` y comprobaciones manuales, sin JUnit

> *Regla de oro del ejercicio: **nunca uses `double` para dinero**. Todo monto es `BigDecimal` con escala 2 y `RoundingMode.HALF_UP`.*
>
> *Restricción del curso: **sin frameworks**. El único archivo externo es el driver JDBC de Oracle, que es una biblioteca de conexión y no un framework. Toda la lógica (autenticación, permisos, transacciones, pantallas) la escribes tú con el JDK.*

---

2. Reglas de negocio (léelas antes de programar)

- **RN-01** Una caja solo puede tener **un turno ABIERTO** a la vez; un cajero solo puede tener **un turno ABIERTO** a la vez.
- **RN-02** No se puede vender sin turno abierto.
- **RN-03** El efectivo inicial es obligatorio (puede ser 0 solo si el supervisor lo autoriza, reto opcional).
- **RN-04** Efectivo esperado en caja = fondo inicial + cobros en efectivo (monto neto, ya descontado el cambio) − reversiones pagadas en efectivo − retiros a bóveda.
- **RN-05** Un retiro a bóveda no puede dejar la caja con menos de un **efectivo mínimo operativo** configurable (ej. Q500).
- **RN-06** Una reversión solo la autoriza un usuario con permiso `REVERSION_AUTORIZAR` y con **nivel de rol estrictamente mayor** al del cajero que la solicita.
- **RN-07** Toda reversión guarda quién la solicitó, quién la autorizó, motivo, fecha y monto.
- **RN-08** Al cerrar el turno el cajero declara el efectivo contado; el sistema calcula la **diferencia** (sobrante o faltante) y bloquea el turno.

- **RN-09** Las contraseñas se almacenan con hash PBKDF2 y sal aleatoria (API estándar del JDK).

---

3. Mapa de pantallas

#	Pantalla	Clase Swing	Permiso requerido
1	Login	`LoginFrame`	Público
2	Selección de caja y apertura de turno	`AperturaTurnoFrame`	`TURNO_ABRIR`
3	Punto de venta (cobro)	`PosFrame`	`VENTA_COBRAR`
4	Pantalla de pago	`PagoDialog`	`VENTA_COBRAR`
5	Corte intermedio	`CorteIntermedioDialog`	`CORTE_INTERMEDIO`
6	Reversión: buscar venta	`ReversionFrame`	`REVERSION_SOLICITAR`
7	Reversión: autorización del supervisor	`AutorizacionDialog`	Credenciales de supervisor
8	Corte final	`CierreTurnoFrame`	`TURNO_CERRAR`
9	Lista y alta de usuarios	`UsuariosFrame`	`USUARIO_ADMIN`
10	Roles y permisos	`RolesFrame`	`ROL_ADMIN`
11	Asignación de roles/ permisos a usuarios	`AsignacionPermisosDialog`	`ROL_ADMIN`
12	Catálogo de productos y cajas (apoyo)	`ProductosFrame`, `CajasFrame`	`CATALOGO_ADMIN`

FASE 0 — Preparación del proyecto

Paso 1 — Instalación. Instala **JDK 21**, **Oracle Database XE 21c** y un IDE (IntelliJ, Eclipse o NetBeans). No se usa Maven ni Gradle.

Paso 2 — Driver JDBC. Descarga `ojdbc11.jar` desde el sitio de Oracle y colócalo en la carpeta `lib/`. Es un driver de conexión, no un framework.

Paso 3 — Usuario de la aplicación. Conectado como `SYSTEM`, crea el dueño de las tablas en la base pluggable `XEPDB1`:

```
ALTER SESSION SET CONTAINER = XEPDB1;
CREATE USER pos IDENTIFIED BY pos123
  DEFAULT TABLESPACE users QUOTA UNLIMITED ON users;
GRANT CREATE SESSION, CREATE TABLE, CREATE SEQUENCE, CREATE VIEW TO pos;
```

Paso 4 — Estructura de carpetas.

```
pos/
├── lib/ojdbc11.jar
├── config.properties        // efectivo mínimo operativo (se lee con
java.util.Properties)
├── sql/                     // 01_esquema.sql, 02_datos_iniciales.sql
└── src/com/supermercado/pos/
    ├── Main.java
    ├── dominio/             // clases del modelo y enums
    ├── dao/                 // acceso a datos con JDBC
    ├── servicio/            // reglas de negocio y transacciones
    ├── ui/                  // ventanas Swing
    ├── util/                // Conexion, Seguridad, Sesion
    └── excepcion/           // ReglaNegocioException, AutorizacionException
```

Paso 5 — Compilar y ejecutar desde la carpeta `pos/` (en Windows usa `;` en lugar de `:` en el classpath):

```
javac -cp lib/ojdbc11.jar -d out $(find src -name "*.java")
java  -cp out:lib/ojdbc11.jar com.supermercado.pos.Main
```

Paso 6 — Clase de conexión (`util/Conexion.java`):

```
public final class Conexion {
    private static final String URL = "jdbc:oracle:thin:@//localhost:1521/
XEPDB1";
    private static final String USUARIO = "pos";
    private static final String CLAVE = "pos123";

    private Conexion() {}

    public static Connection obtener() throws SQLException {
        return DriverManager.getConnection(URL, USUARIO, CLAVE);
    }
}
```

Paso 7 — Prueba de humo (`Main.java`):

```
public class Main {
    public static void main(String[] args) throws Exception {
        try (Connection con = Conexion.obtener();
             ResultSet rs = con.createStatement()
                 .executeQuery("SELECT 'Conectado a Oracle XE' FROM dual")) {
            rs.next();
            System.out.println(rs.getString(1));
        }
    }
}
```

Checkpoint: al ejecutar `Main` se imprime `Conectado a Oracle XE` en la consola.

FASE 1 — Creación del modelo de dominio

1.1 Diagrama conceptual (relaciones)

- `Usuario` N—N `Rol`; `Rol` N—N `Permiso`.
- `Caja` 1—N `Turno`; `Usuario` (cajero) 1—N `Turno`.
- `Turno` 1—N `Venta`; `Turno` 1—N `MovimientoCaja`.
- `Venta` 1—N `VentaDetalle`; `Venta` 1—N `Pago`.
- `VentaDetalle` 1—N `Reversion`.

1.2 Seguridad: Usuario, Rol, Permiso

```
// Clases simples (POJO): atributos privados + getters/setters generados por
el IDE.
// No hay anotaciones ni mapeo automático: los DAO arman estos objetos desde
el ResultSet.
// Implementa equals/hashCode (por id o por codigo) en Permiso y Rol para
usarlos en Set.
public class Permiso {
    private Long id;
    private String codigo;        // ej. "VENTA_COBRAR"
    private String descripcion;
}

public class Rol {
    private Long id;
    private String nombre;        // CAJERO, SUPERVISOR, GERENTE, ADMIN
    private int nivel;            // 1=cajero, 2=supervisor, 3=gerente...
    private Set<Permiso> permisos = new HashSet<>();
}

public class Usuario {
    private Long id;
    private String username;
    private String passwordHash;
    private String nombreCompleto;
    private boolean activo = true;
    private Set<Rol> roles = new HashSet<>();
    private Set<Permiso> permisosExtra = new HashSet<>();   // asignados
directamente

    public int nivelMaximo() {
        return roles.stream().mapToInt(Rol::getNivel).max().orElse(0);
    }

    public boolean tienePermiso(String codigo) {
        return roles.stream().flatMap(r -> r.getPermisos().stream())
                    .anyMatch(p -> p.getCodigo().equals(codigo))
            || permisosExtra.stream().anyMatch(p ->
p.getCodigo().equals(codigo));
```

```
    }
}
```

1.3 Caja y Turno

```
public class Caja {
    private Long id;
    private Integer numero;
    private String nombre;
    private boolean activa = true;
}

public enum EstadoTurno { ABIERTO, CERRADO }

public class Turno {
    private Long id;
    private Caja caja;
    private Usuario cajero;
    private LocalDateTime apertura;
    private LocalDateTime cierre;
    private BigDecimal fondoInicial;
    private BigDecimal efectivoDeclarado;
    private BigDecimal efectivoEsperado;
    private BigDecimal diferencia;
    private EstadoTurno estado = EstadoTurno.ABIERTO;
}
```

> ***Ejercicio 1.A:*** *garantiza RN-01 a nivel de base de datos. Oracle no tiene índices únicos parciales, pero un índice único basado en una expresión ignora los valores NULL, y con eso basta (el script de la sección 1.6 ya trae el índice de la caja). Crea tú el índice equivalente para* `cajero_id` *y, en el servicio, captura la* `SQLException` *con* `getErrorCode() == 1` *(ORA-00001) para mostrar un mensaje claro.*

1.4 Movimientos de caja

```
public enum TipoMovimiento { FONDO_INICIAL, RETIRO_BOVEDA, REVERSION_EFECTIVO
}

public class MovimientoCaja {
    private Long id;
    private Turno turno;
    private TipoMovimiento tipo;
    private BigDecimal monto;
    private Usuario autorizadoPor;   // supervisor que recibe el dinero o
autoriza
    private String referencia;       // folio de recibo de bóveda
    private LocalDateTime fecha = LocalDateTime.now();

    public static MovimientoCaja de(Turno t, TipoMovimiento tipo,
                                    BigDecimal monto, Usuario autoriza) { /
* ... */ }
}
```

1.5 Productos, ventas, pagos y reversiones

```
public class Producto {
    private Long id;
    private String codigoBarras;
    private String nombre;
    private BigDecimal precio;
    private BigDecimal porcentajeImpuesto;   // 12.00
    private int stock;
    private boolean activo = true;
}

public enum EstadoVenta { COMPLETADA, PARCIALMENTE_REVERSADA, ANULADA }

public class Venta {
    private Long id;
    private Turno turno;
    private LocalDateTime fecha = LocalDateTime.now();
    private List<VentaDetalle> detalles = new ArrayList<>();
    private List<Pago> pagos = new ArrayList<>();
    private BigDecimal total;
    private EstadoVenta estado = EstadoVenta.COMPLETADA;
}

public class VentaDetalle {
    private Long id;
    private Venta venta;
    private Producto producto;
    private int cantidad;
    private int cantidadReversada = 0;
    private BigDecimal precioUnitario;   // congelado al vender
    private BigDecimal subtotal;
}

public enum MetodoPago { EFECTIVO, TARJETA }

public class Pago {
    private Long id;
    private Venta venta;
    private MetodoPago metodo;
    private BigDecimal monto;       // aplicado a la venta
    private BigDecimal recibido;    // lo que entregó el cliente
    private BigDecimal cambio;
}

public class Reversion {
    private Long id;
    private VentaDetalle detalle;
    private int cantidad;
    private BigDecimal monto;
    private String motivo;
    private Usuario solicitadoPor;
    private Usuario autorizadoPor;
```

```
    private LocalDateTime fecha = LocalDateTime.now();
}
```

> ***Pregunta de reflexión 1:*** *¿por qué `VentaDetalle` guarda `precioUnitario` en lugar de leer siempre `Producto.precio`?*

1.6 Esquema físico en Oracle XE (`sql/01_esquema.sql`)

Ejecútalo conectado como el usuario `pos`. Las llaves primarias usan columnas de identidad (Oracle 12c o superior) y todo monto es `NUMBER(12,2)`.

```
CREATE TABLE permiso (
  id          NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  codigo      VARCHAR2(50) NOT NULL UNIQUE,
  descripcion VARCHAR2(200)
);
CREATE TABLE rol (
  id     NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  nombre VARCHAR2(50) NOT NULL UNIQUE,
  nivel  NUMBER(2) NOT NULL
);
CREATE TABLE rol_permiso (
  rol_id     NUMBER NOT NULL REFERENCES rol(id),
  permiso_id NUMBER NOT NULL REFERENCES permiso(id),
  PRIMARY KEY (rol_id, permiso_id)
);
CREATE TABLE usuario (
  id              NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  username        VARCHAR2(50) NOT NULL UNIQUE,
  password_hash   VARCHAR2(200) NOT NULL,
  nombre_completo VARCHAR2(120),
  activo          NUMBER(1) DEFAULT 1 NOT NULL
);
CREATE TABLE usuario_rol (
  usuario_id NUMBER NOT NULL REFERENCES usuario(id),
  rol_id     NUMBER NOT NULL REFERENCES rol(id),
  PRIMARY KEY (usuario_id, rol_id)
);
CREATE TABLE usuario_permiso (
  usuario_id NUMBER NOT NULL REFERENCES usuario(id),
  permiso_id NUMBER NOT NULL REFERENCES permiso(id),
  PRIMARY KEY (usuario_id, permiso_id)
);
CREATE TABLE caja (
  id     NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  numero NUMBER(4) NOT NULL UNIQUE,
  nombre VARCHAR2(60),
  activa NUMBER(1) DEFAULT 1 NOT NULL
);
CREATE TABLE turno (
  id                NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  caja_id           NUMBER NOT NULL REFERENCES caja(id),
  cajero_id         NUMBER NOT NULL REFERENCES usuario(id),
  apertura          TIMESTAMP NOT NULL,
```

```
  cierre             TIMESTAMP,
  fondo_inicial      NUMBER(12,2) NOT NULL,
  efectivo_declarado NUMBER(12,2),
  efectivo_esperado  NUMBER(12,2),
  diferencia         NUMBER(12,2),
  estado             VARCHAR2(10) DEFAULT 'ABIERTO' NOT NULL
                     CHECK (estado IN ('ABIERTO','CERRADO'))
);
-- RN-01: una sola caja abierta por vez (Oracle ignora los NULL en índices
únicos)
CREATE UNIQUE INDEX ux_turno_caja_abierto
  ON turno (CASE WHEN estado = 'ABIERTO' THEN caja_id END);

CREATE TABLE movimiento_caja (
  id             NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  turno_id       NUMBER NOT NULL REFERENCES turno(id),
  tipo           VARCHAR2(20) NOT NULL
                 CHECK (tipo IN
('FONDO_INICIAL','RETIRO_BOVEDA','REVERSION_EFECTIVO')),
  monto          NUMBER(12,2) NOT NULL CHECK (monto >= 0),
  autorizado_por NUMBER REFERENCES usuario(id),
  referencia     VARCHAR2(40),
  fecha          TIMESTAMP DEFAULT SYSTIMESTAMP NOT NULL
);
CREATE TABLE producto (
  id                  NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  codigo_barras       VARCHAR2(30) NOT NULL UNIQUE,
  nombre              VARCHAR2(120) NOT NULL,
  precio              NUMBER(12,2) NOT NULL,
  porcentaje_impuesto NUMBER(5,2) DEFAULT 12 NOT NULL,
  stock               NUMBER(8) DEFAULT 0 NOT NULL CHECK (stock >= 0),
  activo              NUMBER(1) DEFAULT 1 NOT NULL
);
CREATE TABLE venta (
  id       NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  turno_id NUMBER NOT NULL REFERENCES turno(id),
  fecha    TIMESTAMP DEFAULT SYSTIMESTAMP NOT NULL,
  total    NUMBER(12,2) NOT NULL,
  estado   VARCHAR2(25) DEFAULT 'COMPLETADA' NOT NULL
           CHECK (estado IN
('COMPLETADA','PARCIALMENTE_REVERSADA','ANULADA'))
);
CREATE TABLE venta_detalle (
  id                 NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  venta_id           NUMBER NOT NULL REFERENCES venta(id),
  producto_id        NUMBER NOT NULL REFERENCES producto(id),
  cantidad           NUMBER(6) NOT NULL CHECK (cantidad > 0),
  cantidad_reversada NUMBER(6) DEFAULT 0 NOT NULL,
  precio_unitario    NUMBER(12,2) NOT NULL,
  subtotal           NUMBER(12,2) NOT NULL
);
CREATE TABLE pago (
  id       NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  venta_id NUMBER NOT NULL REFERENCES venta(id),
```

```
  metodo   VARCHAR2(10) NOT NULL CHECK (metodo IN ('EFECTIVO','TARJETA')),
  monto    NUMBER(12,2) NOT NULL,
  recibido NUMBER(12,2) NOT NULL,
  cambio   NUMBER(12,2) DEFAULT 0 NOT NULL
);
CREATE TABLE reversion (
  id             NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  detalle_id     NUMBER NOT NULL REFERENCES venta_detalle(id),
  cantidad       NUMBER(6) NOT NULL,
  monto          NUMBER(12,2) NOT NULL,
  motivo         VARCHAR2(200) NOT NULL,
  solicitado_por NUMBER NOT NULL REFERENCES usuario(id),
  autorizado_por NUMBER NOT NULL REFERENCES usuario(id),
  fecha          TIMESTAMP DEFAULT SYSTIMESTAMP NOT NULL
);
```

FASE 2 — DAO con JDBC y datos iniciales

Crea un DAO por entidad (`TurnoDAO`, `UsuarioDAO`, `ProductoDAO`...). Reglas: cada método recibe la `Connection`, para que el servicio controle la transacción con `commit` / `rollback`; usa siempre `PreparedStatement` con parámetros `?` (evita la inyección SQL) y cierra los recursos con try-with-resources. Métodos clave:

```
public class TurnoDAO {

    public Optional<Turno> buscarAbiertoPorCajero(Connection con, long
cajeroId) throws SQLException {
        String sql = "SELECT * FROM turno WHERE cajero_id = ? AND estado =
'ABIERTO'";
        try (PreparedStatement ps = con.prepareStatement(sql)) {
            ps.setLong(1, cajeroId);
            try (ResultSet rs = ps.executeQuery()) {
                return rs.next() ? Optional.of(mapear(con, rs)) :
Optional.empty();
            }
        }
    }

    public long insertar(Connection con, Turno t) throws SQLException {
        String sql = "INSERT INTO turno (caja_id, cajero_id, apertura,
fondo_inicial, estado) "
                   + "VALUES (?, ?, ?, ?, 'ABIERTO')";
        // En Oracle se pide la columna ID para recuperar la llave generada
        try (PreparedStatement ps = con.prepareStatement(sql, new String[]
{"ID"})) {
            ps.setLong(1, t.getCaja().getId());
            ps.setLong(2, t.getCajero().getId());
            ps.setTimestamp(3, Timestamp.valueOf(t.getApertura()));
            ps.setBigDecimal(4, t.getFondoInicial());
            ps.executeUpdate();
            try (ResultSet k = ps.getGeneratedKeys()) {
                k.next();
```

```
                return k.getLong(1);
            }
        }
    }

    private Turno mapear(Connection con, ResultSet rs) throws SQLException {
        // arma el Turno columna por columna; carga Caja y Usuario con sus
DAO
    }
}

public class UsuarioDAO {
    // Carga el usuario con sus roles, los permisos de cada rol y sus
permisos extra
    public Optional<Usuario> buscarPorUsername(Connection con, String
username) throws SQLException { /* ... */ }
}

public class ProductoDAO {
    // Descuento atómico: solo actúa si hay existencias suficientes
    public int descontarStock(Connection con, long productoId, int cantidad)
throws SQLException {
        String sql = "UPDATE producto SET stock = stock - ? WHERE id = ? AND
stock >= ?";
        try (PreparedStatement ps = con.prepareStatement(sql)) {
            ps.setInt(1, cantidad);
            ps.setLong(2, productoId);
            ps.setInt(3, cantidad);
            return ps.executeUpdate();     // 0 = no había stock suficiente
        }
    }
}
```

Datos iniciales (`sql/02_datos_iniciales.sql`): inserta los permisos de la tabla del mapa de pantallas más `REVERSION_SOLICITAR` y `REVERSION_AUTORIZAR`, los roles `CAJERO (nivel 1)`, `SUPERVISOR (2)`, `GERENTE (3)` y `ADMIN (4)`, tres cajas, productos de prueba y el usuario `admin` / `Admin123!`. Como el hash PBKDF2 se calcula en Java (Fase 3), genera el del administrador con una clase de apoyo `GenerarHash` y pega el resultado en el script.

FASE 3 — Pantalla de autenticación (Login)

3.1 Hash de contraseñas con la API estándar del JDK

Sin librerías externas: usa PBKDF2 (`javax.crypto`) con una sal aleatoria por usuario (`util/Seguridad.java`). Se guarda en la columna `password_hash` con el formato `iteraciones:sal:hash`.

```
public final class Seguridad {
    private static final int ITERACIONES = 120_000;
    private static final int LONGITUD_BITS = 256;
```

```
    private Seguridad() {}

    public static String hash(char[] clave) {
        byte[] sal = new byte[16];
        new SecureRandom().nextBytes(sal);
        byte[] h = pbkdf2(clave, sal, ITERACIONES);
        return ITERACIONES + ":" + Base64.getEncoder().encodeToString(sal)
                           + ":" + Base64.getEncoder().encodeToString(h);
    }

    public static boolean verificar(char[] clave, String almacenado) {
        String[] p = almacenado.split(":");
        byte[] esperado = Base64.getDecoder().decode(p[2]);
        byte[] calculado = pbkdf2(clave, Base64.getDecoder().decode(p[1]),
Integer.parseInt(p[0]));
        return MessageDigest.isEqual(esperado, calculado);   // comparación
en tiempo constante
    }

    private static byte[] pbkdf2(char[] clave, byte[] sal, int iteraciones) {
        try {
            PBEKeySpec spec = new PBEKeySpec(clave, sal, iteraciones,
LONGITUD_BITS);
            return SecretKeyFactory.getInstance("PBKDF2WithHmacSHA256")
                                   .generateSecret(spec).getEncoded();
        } catch (GeneralSecurityException e) {
            throw new IllegalStateException(e);
        }
    }
}
```

3.2 Servicio de autenticación y sesión

El servicio valida las credenciales contra Oracle y devuelve el `Usuario` con sus roles y permisos. La clase `Sesion` recuerda quién está operando la caja mientras la aplicación esté abierta (es una aplicación de escritorio, no hay sesiones HTTP).

```
public class AuthService {
    private final UsuarioDAO usuarioDao = new UsuarioDAO();

    public Usuario autenticar(String username, char[] clave) {
        try (Connection con = Conexion.obtener()) {
            return usuarioDao.buscarPorUsername(con, username)
                .filter(Usuario::isActivo)
                .filter(u -> Seguridad.verificar(clave, u.getPasswordHash()))
                .orElseThrow(() -> new AutorizacionException("Usuario o
contraseña inválidos"));
        } catch (SQLException e) {
            throw new IllegalStateException("Error de base de datos", e);
        }
    }
}
```

```
public final class Sesion {
    private static Usuario usuario;
    private static Turno turno;
    // getters y setters estáticos; cerrar() deja ambos en null
}
```

3.3 Ventana `LoginFrame`

Un `JFrame` con `JTextField` (usuario), `JPasswordField` (contraseña), un `JButton` «Ingresar» marcado como botón por defecto (`getRootPane().setDefaultButton(...)`, para que Enter lo active) y un `JLabel` para los errores. Usa `getPassword()`, que devuelve un `char[]`, y bórralo al terminar:

```
btnIngresar.addActionListener(e -> {
    char[] clave = txtClave.getPassword();
    try {
        Usuario u = authService.autenticar(txtUsuario.getText().trim(),
clave);
        Sesion.setUsuario(u);
        dispose();
        new MenuPrincipalFrame().setVisible(true);
    } catch (AutorizacionException ex) {
        lblError.setText(ex.getMessage());
    } finally {
        Arrays.fill(clave, '\0');     // no dejes la contraseña en memoria
        txtClave.setText("");
    }
});
```

3.4 Menú principal y navegación por permisos

Tras el login se abre `MenuPrincipalFrame`, con un botón por cada pantalla. Cada botón se habilita solo si `Sesion.getUsuario().tienePermiso(...)` es verdadero. Si el usuario tiene `VENTA_COBRAR`: con turno abierto va directo a `PosFrame` y sin turno a `AperturaTurnoFrame`. Crea toda ventana dentro de `SwingUtilities.invokeLater(...)`; en `Main` reemplaza la prueba de humo por `SwingUtilities.invokeLater(() -> new LoginFrame().setVisible(true));`.

Checkpoint: inicia sesión con `admin`; con un usuario inactivo debe rechazarse.

FASE 4 — Apertura de turno con efectivo inicial

4.1 Ventana `AperturaTurnoFrame`

Campos: **caja** (`JComboBox` con las cajas activas que no tienen turno abierto) y **efectivo inicial** (`JTextField` convertido a `BigDecimal`; captura `NumberFormatException` si el texto no es un número). Se muestra un resumen: cajero, fecha y hora.

4.2 Servicio

La apertura es una transacción: o se crean el turno y su movimiento de fondo inicial, o no se crea nada.

```
public Turno abrir(Usuario cajero, long cajaId, BigDecimal fondo) {
    if (fondo == null || fondo.signum() <= 0)
        throw new ReglaNegocioException("Debe declarar el efectivo inicial");
    try (Connection con = Conexion.obtener()) {
        con.setAutoCommit(false);
        try {
            if (turnoDao.buscarAbiertoPorCajero(con,
cajero.getId()).isPresent())
                throw new ReglaNegocioException("Ya tienes un turno
abierto");
            Caja caja = cajaDao.buscarPorId(con, cajaId).orElseThrow();
            if (turnoDao.existeAbiertoEnCaja(con, cajaId))
                throw new ReglaNegocioException("La caja está ocupada por
otro cajero");

            Turno t = new Turno();
            t.setCaja(caja);
            t.setCajero(cajero);
            t.setApertura(LocalDateTime.now());
            t.setFondoInicial(fondo.setScale(2, RoundingMode.HALF_UP));
            t.setId(turnoDao.insertar(con, t));
            movimientoDao.insertar(con,
                MovimientoCaja.de(t, TipoMovimiento.FONDO_INICIAL,
t.getFondoInicial(), null));
            con.commit();
            return t;
        } catch (SQLException | RuntimeException e) {
            con.rollback();
            if (e instanceof SQLException s && s.getErrorCode() == 1)   //
ORA-00001: índice único
                throw new ReglaNegocioException("La caja o el cajero ya
tienen un turno abierto");
            throw e;
        }
    } catch (SQLException e) {
        throw new IllegalStateException(e);
    }
}
```

> ***Ejercicio 4.A:*** *al abrir con éxito, guarda el turno con* `Sesion.setTurno(t)` *y abre* `PosFrame`*. Captura* `ReglaNegocioException` *y muestra su mensaje con* `JOptionPane.showMessageDialog(this, mensaje, "Aviso", JOptionPane.ERROR_MESSAGE)`*.*

FASE 5 — Pantallas de cobro

5.1 Ventana `PosFrame` (carrito)

- `JTextField` para capturar el **código de barras**, con el foco siempre en él (`requestFocusInWindow()`); un lector de código de barras se comporta como un teclado que termina con Enter, así que basta un `ActionListener`.
- `JTable` con un `DefaultTableModel` para el carrito: producto, cantidad (editable), precio, subtotal; un botón para quitar la fila seleccionada.
- Etiquetas con subtotal, impuesto y **total a pagar** (supuesto: el precio ya incluye el impuesto, que solo se desglosa en el ticket).
- Botones: *Cobrar* (atajo F12 con `KeyStroke`), *Cancelar venta*, *Reversión*, *Corte intermedio* y *Cerrar turno*.

El carrito vive en memoria dentro de la ventana hasta cobrar:

```
public record LineaCarrito(Producto producto, int cantidad) {
    public BigDecimal subtotal() {
        return producto.getPrecio().multiply(BigDecimal.valueOf(cantidad));
    }
    public LineaCarrito sumar(LineaCarrito otra) {
        return new LineaCarrito(producto, cantidad + otra.cantidad);
    }
}

public class Carrito {
    private final Map<Long, LineaCarrito> lineas = new LinkedHashMap<>();

    public void agregar(Producto p, int cantidad) {
        lineas.merge(p.getId(), new LineaCarrito(p, cantidad),
LineaCarrito::sumar);
    }
    public Collection<LineaCarrito> lineas() { return lineas.values(); }
    public void vaciar() { lineas.clear(); }

    public BigDecimal total() {
        return lineas.values().stream().map(LineaCarrito::subtotal)
                .reduce(BigDecimal.ZERO, BigDecimal::add).setScale(2,
RoundingMode.HALF_UP);
    }
}
```

5.2 Ventana `PagoDialog`

Un `JDialog` modal que permite **pago en efectivo**, **tarjeta** o **mixto**. Para efectivo, captura el monto recibido y calcula el **cambio** automáticamente (con un `DocumentListener` sobre el campo de texto).

```
public Venta cobrar(Turno turno, Carrito carrito, List<PagoDto> pagos) {
    BigDecimal total = carrito.total();
    BigDecimal pagado =
```

```
pagos.stream().map(PagoDto::monto).reduce(BigDecimal.ZERO, BigDecimal::add);
    if (pagado.compareTo(total) < 0) throw new ReglaNegocioException("Pago
insuficiente");

    try (Connection con = Conexion.obtener()) {
        con.setAutoCommit(false);
        try {
            Venta v = new Venta();
            v.setTurno(turno);
            v.setTotal(total);
            v.setId(ventaDao.insertar(con, v));

            for (LineaCarrito l : carrito.lineas()) {
                if (productoDao.descontarStock(con, l.producto().getId(),
l.cantidad()) == 0)
                    throw new ReglaNegocioException("Sin stock: " +
l.producto().getNombre());
                ventaDetalleDao.insertar(con, v, l);        // congela el
precio unitario
            }
            // Pagos: para EFECTIVO, monto aplicado = recibido - cambio.
            // El cambio solo puede salir de pagos en efectivo.
            for (PagoDto p : pagos) pagoDao.insertar(con, v, p);

            con.commit();
            return v;
        } catch (SQLException | RuntimeException e) {
            con.rollback();          // sin venta, sin pagos y sin cambio en
el stock
            throw e;
        }
    } catch (SQLException e) {
        throw new IllegalStateException(e);
    }
}
```

Después de cobrar, muestra el **ticket** en un `JDialog` con un `JTextArea` de fuente monoespaciada (`Font.MONOSPACED`): folio, detalle, total, pago y cambio. Luego vacía el carrito.

> ***Pregunta de reflexión 2:*** *¿qué ocurre si dos cajas venden el último producto en el mismo instante? Analiza la sentencia* `UPDATE producto SET stock = stock - ? WHERE id = ? AND stock >= ?`*: ¿por qué es segura y qué devuelve* `executeUpdate()` *a la caja que llegó segunda?*

FASE 6 — Corte de caja intermedio (retiro a bóveda)

Objetivo: reducir el efectivo acumulado en la caja por seguridad.

6.1 Ventana `CorteIntermedioDialog`

Muestra, calculado por el sistema:

- Fondo inicial
- (+) Ventas en efectivo (neto)
- (−) Reversiones en efectivo
- (−) Retiros anteriores
- **= Efectivo actual esperado en caja**

El cajero captura el **monto a entregar** y el **supervisor/tesorero** que lo recibe debe confirmar con sus credenciales (mismo mecanismo de la Fase 7). Se imprime un **recibo de retiro** con folio.

6.2 Servicio

```
public MovimientoCaja retirarABoveda(Turno turno, BigDecimal monto, Usuario
receptor) {
    if (monto == null || monto.signum() <= 0)
        throw new ReglaNegocioException("Monto inválido");
    try (Connection con = Conexion.obtener()) {
        con.setAutoCommit(false);
        try {
            BigDecimal actual = calculadora.efectivoEsperado(con, turno);
            // efectivoMinimo se lee de config.properties con
java.util.Properties
            if (actual.subtract(monto).compareTo(efectivoMinimo) < 0)
                throw new ReglaNegocioException("La caja debe conservar al
menos " + efectivoMinimo);

            MovimientoCaja m = MovimientoCaja.de(turno,
TipoMovimiento.RETIRO_BOVEDA, monto, receptor);
            m.setId(movimientoDao.insertar(con, m));
            con.commit();
            return m;
        } catch (SQLException | RuntimeException e) {
            con.rollback();
            throw e;
        }
    } catch (SQLException e) {
        throw new IllegalStateException(e);
    }
}
```

6.3 Calculadora de efectivo (reutilizada en el corte final)

```
public BigDecimal efectivoEsperado(Connection con, Turno t) throws
SQLException {
    BigDecimal fondo   = t.getFondoInicial();
    BigDecimal cobros  = escalar(con,
        "SELECT NVL(SUM(p.monto), 0) FROM pago p JOIN venta v ON v.id =
p.venta_id "
      + "WHERE v.turno_id = ? AND p.metodo = 'EFECTIVO'", t.getId());
    BigDecimal retiros = sumaMovimientos(con, t,
TipoMovimiento.RETIRO_BOVEDA);
    BigDecimal revers  = sumaMovimientos(con, t,
```

```
TipoMovimiento.REVERSION_EFECTIVO);
    return fondo.add(cobros).subtract(retiros).subtract(revers);
}
```

> ***Ejercicio 6.A:*** *implementa `escalar` y `sumaMovimientos` con `PreparedStatement` (esta última suma `monto` de `movimiento_caja` por turno y tipo) y cubre con una prueba el caso de pago mixto.*

FASE 7 — Reversión de producto con autorización de un rol superior

Esta es la parte más delicada del diseño. El cajero **conserva su sesión**; solo se **valida puntualmente** a otro usuario sin cambiar el contexto de seguridad.

7.1 Ventana `ReversionFrame` (solicitud)

1. El cajero busca la venta por **folio** (solo ventas de su turno actual; ventas de otros turnos podrían requerir permisos extra, reto opcional).
2. Se muestra el detalle con la cantidad que aún puede reversarse (`cantidad − cantidadReversada`).
3. Selecciona el producto, la **cantidad** y escribe el **motivo** (obligatorio).

7.2 Ventana `AutorizacionDialog` (credenciales del supervisor)

Es un `JDialog` modal que muestra el resumen de la reversión (producto, cantidad, monto, cajero solicitante) y pide **usuario y contraseña del supervisor**. No es un login: la sesión del cajero (`Sesion`) no cambia; el diálogo solo devuelve el `Usuario` validado, o `null` si se cancela.

7.3 Servicio de autorización

```
public class AutorizacionService {
    private final UsuarioDAO usuarioDao = new UsuarioDAO();

    public Usuario validarSupervisor(String username, char[] clave, Usuario
solicitante) {
        try (Connection con = Conexion.obtener()) {
            Usuario sup = usuarioDao.buscarPorUsername(con, username)
                .filter(Usuario::isActivo)
                .filter(u -> Seguridad.verificar(clave, u.getPasswordHash()))
                .orElseThrow(() -> new AutorizacionException("Credenciales
inválidas"));

            if (!sup.tienePermiso("REVERSION_AUTORIZAR"))
                throw new AutorizacionException("El usuario no puede
autorizar reversiones");
            if (sup.nivelMaximo() <= solicitante.nivelMaximo())
                throw new AutorizacionException("Se requiere un rol superior
al del solicitante");
            if (sup.getId().equals(solicitante.getId()))
                throw new AutorizacionException("No puedes autorizarte a ti
```

```
mismo");
            return sup;
        } catch (SQLException e) {
            throw new IllegalStateException(e);
        }
    }
}
```

7.4 Servicio de reversión

```
public Reversion revertir(long detalleId, int cantidad, String motivo,
                          Turno turno, Usuario cajero, Usuario supervisor) {
    if (motivo == null || motivo.isBlank())
        throw new ReglaNegocioException("El motivo es obligatorio");
    try (Connection con = Conexion.obtener()) {
        con.setAutoCommit(false);
        try {
            // SELECT ... FOR UPDATE: bloquea la fila hasta el commit
            VentaDetalle d = detalleDao.buscarParaActualizar(con, detalleId);
            if (cantidad <= 0 || cantidad > d.getCantidad() -
d.getCantidadReversada())
                throw new ReglaNegocioException("Cantidad no reversable");

            BigDecimal monto = d.getPrecioUnitario()
                    .multiply(BigDecimal.valueOf(cantidad)).setScale(2,
RoundingMode.HALF_UP);

            detalleDao.sumarReversado(con, detalleId, cantidad);
            productoDao.reingresarStock(con, d.getProducto().getId(),
cantidad);
            ventaDao.actualizarEstado(con, d.getVenta().getId());   //
PARCIALMENTE_REVERSADA / ANULADA

            Reversion r = Reversion.de(d, cantidad, monto, motivo, cajero,
supervisor);
            r.setId(reversionDao.insertar(con, r));
            // El dinero devuelto al cliente sale de la caja:
            movimientoDao.insertar(con,
                MovimientoCaja.de(turno, TipoMovimiento.REVERSION_EFECTIVO,
monto, supervisor));
            con.commit();
            return r;
        } catch (SQLException | RuntimeException e) {
            con.rollback();
            throw e;
        }
    } catch (SQLException e) {
        throw new IllegalStateException(e);
    }
}
```

7.5 Reglas de seguridad adicionales

- Limita los **intentos fallidos** de credenciales de supervisor (ej. 3 intentos y registro en bitácora).

- Registra en una tabla de **auditoría** cada intento (exitoso o no).
- Nunca guardes la contraseña del supervisor en un campo ni en la clase Sesion; limpia el arreglo char[] con Arrays.fill al terminar.

> ***Pregunta de reflexión 3:*** *¿por qué es preferible re-autenticar con credenciales que pedir un simple "PIN compartido" o un checkbox de "autorizado"?*

FASE 8 — Corte de caja final (cierre de turno)

8.1 Ventana `CierreTurnoFrame`

1. Muestra el **resumen del turno**: número de ventas, total vendido, ventas por método de pago, reversiones, retiros a bóveda.
2. El cajero captura el **efectivo contado**. Idealmente una pequeña tabla de **denominaciones** (billetes y monedas × cantidad) que suma automáticamente.
3. Al confirmar, el sistema calcula y muestra la **diferencia**.
4. Se imprime el **reporte de cierre** y el turno queda `CERRADO`.

> *Buena práctica: en cajas reales el cajero declara su monto **antes** de ver el esperado (cierre "ciego") para evitar ajustes. Implementa esa variante como mejora.*

8.2 Servicio

```
public Turno cerrar(Turno t, BigDecimal contado) {
    try (Connection con = Conexion.obtener()) {
        con.setAutoCommit(false);
        try {
            if (!turnoDao.estaAbierto(con, t.getId()))
                throw new ReglaNegocioException("El turno ya está cerrado");
            BigDecimal esperado = calculadora.efectivoEsperado(con, t);
            t.setEfectivoEsperado(esperado);
            t.setEfectivoDeclarado(contado);
            t.setDiferencia(contado.subtract(esperado));   // + sobrante / -
faltante
            t.setCierre(LocalDateTime.now());
            t.setEstado(EstadoTurno.CERRADO);
            turnoDao.cerrar(con, t);
            con.commit();
            return t;
        } catch (SQLException | RuntimeException e) {
            con.rollback();
            throw e;
        }
    } catch (SQLException e) {
        throw new IllegalStateException(e);
    }
}
```

Al cerrar, limpia el turno de la sesión (`Sesion.setTurno(null)`) y regresa al `MenuPrincipalFrame` o al login para el siguiente cajero. Una vez cerrado, ninguna venta ni reversión puede ligarse a ese turno (RN-02).

FASE 9 — Administración: usuarios, roles y permisos

9.1 Usuarios (`UsuariosFrame`)

- Listado en un `JTable` con búsqueda (un `JTextField` que filtra las filas), estado (activo/inactivo) y roles.
- Formulario de alta/edición en un `JDialog`: username (único), nombre, contraseña inicial (se guarda con `Seguridad.hash`) y rol.
- **Nunca se borran usuarios** con historial: se desactivan (`activo = 0`).
- Validaciones escritas por ti en el servicio: contraseña de mínimo 8 caracteres y username sin espacios.

9.2 Roles y permisos (`RolesFrame`)

- Crear rol con **nombre** y **nivel jerárquico**.
- Matriz de permisos con casillas de verificación (cada permiso agrupado por módulo: Venta, Turno, Reversión, Administración).

9.3 Asignación de permisos a un usuario (`AsignacionPermisosDialog`)

- Selecciona uno o varios roles del usuario.
- Muestra los **permisos efectivos** (los del rol, en gris) y permite sumar `permisosExtra` puntuales.
- Reglas: un usuario no puede asignarse roles de nivel igual o superior al suyo, ni modificar su propio rol (evita escalamiento de privilegios).

```
public void asignarRoles(long usuarioId, Set<Long> rolIds, Usuario actor) {
    // Sin framework: la autorización es una verificación explícita en el
servicio
    if (!actor.tienePermiso("ROL_ADMIN"))
        throw new AutorizacionException("No tienes permiso para administrar
roles");
    if (usuarioId == actor.getId())
        throw new ReglaNegocioException("No puedes modificar tus propios
roles");
    try (Connection con = Conexion.obtener()) {
        con.setAutoCommit(false);
        try {
            for (Rol r : rolDao.buscarPorIds(con, rolIds))
                if (r.getNivel() >= actor.nivelMaximo())
                    throw new ReglaNegocioException("No puedes asignar un rol
```

```
igual o superior al tuyo");
            // DELETE de los roles actuales + INSERT por lotes (addBatch /
executeBatch)
            usuarioDao.reemplazarRoles(con, usuarioId, rolIds);
            con.commit();
        } catch (SQLException | RuntimeException e) {
            con.rollback();
            throw e;
        }
    } catch (SQLException e) {
        throw new IllegalStateException(e);
    }
}
```

> *Los cambios de permisos de un usuario con sesión activa aplican en su próximo inicio de sesión (reto: invalida la sesión de ese usuario).*

FASE 10 — Pruebas

Escribe como mínimo estas pruebas en una clase Pruebas con su propio método verificar(...), sin JUnit ni Mockito:

Caso	Resultado esperado
Abrir turno con caja ocupada	`ReglaNegocioException`
Abrir turno con fondo 0	Rechazado
Cobro con pago insuficiente	Rechazado, sin venta ni cambio de stock
Pago mixto efectivo + tarjeta	Cambio solo sobre el efectivo
Retiro que deja la caja bajo el mínimo	Rechazado
Reversión autorizada por un cajero	`AutorizacionException`
Reversión autorizada por el mismo usuario	Rechazada
Reversión parcial dos veces que supera la cantidad vendida	Rechazada en la segunda
Cierre con faltante de Q20	`diferencia = -20.00`

Estructura mínima del ejecutor de pruebas (`Pruebas.java`, con su propio `main`):

```
public class Pruebas {
    private static int fallos = 0;

    static void verificar(boolean condicion, String caso) {
        System.out.println((condicion ? "OK     " : "FALLÓ  ") + caso);
        if (!condicion) fallos++;
    }

    static void esperaExcepcion(Class<? extends RuntimeException> tipo,
Runnable accion, String caso) {
```

```
        try {
            accion.run();
            verificar(false, caso + " (no lanzó excepción)");
        } catch (RuntimeException e) {
            verificar(tipo.isInstance(e), caso);
        }
    }

    public static void main(String[] args) {
        esperaExcepcion(ReglaNegocioException.class,
            () -> turnoService.abrir(ana, 1L, BigDecimal.ZERO), "Abrir turno
con fondo 0");
        // ... resto de los casos de la tabla
        System.exit(fallos == 0 ? 0 : 1);
    }
}
```

Las pruebas que tocan la base de datos se ejecutan contra un esquema aparte (por ejemplo, el usuario `pos_test` con el mismo script `01_esquema.sql`), para no ensuciar los datos reales.

FASE 11 — Escenario de validación integral

Ejecuta este guion completo y verifica los resultados contra tus cálculos:

1. Admin crea usuarios `ana` (CAJERO), `luis` (CAJERO), `marta` (SUPERVISOR).
2. `ana` inicia sesión y abre la **Caja 1** con **Q1,000.00**.
3. `ana` cobra tres ventas: Q150.00 (efectivo, recibe Q200), Q80.00 (tarjeta) y Q120.00 (mixta: Q50 tarjeta + Q70 efectivo).
4. `luis` intenta abrir la Caja 1 → debe fallar. Abre la Caja 2 con Q800.00.
5. `ana` hace un corte intermedio y entrega **Q500.00** a `marta`.
6. `ana` revierte 1 producto de Q30.00 de la venta de Q150: `marta` debe autorizar; intenta primero con `luis` (debe fallar).
7. `ana` cierra su turno contando el efectivo.

Efectivo esperado de Ana: 1,000 + 150 + 70 − 500 − 30 = **Q690.00**. Si cuentas Q680.00, la diferencia debe ser **−Q10.00**.

---

12. Retos de ampliación

1. Impresión de tickets y reportes con `java.awt.print`.
2. Separar cliente y servidor con sockets o `com.sun.net.httpserver` (ambos incluidos en el JDK), para que varias cajas compartan un solo servicio.

3. Bitácora de auditoría con triggers de Oracle (PL/SQL).
4. Descuentos y promociones con autorización de supervisor.
5. Reporte gerencial en SQL: ventas por caja/cajero/hora, faltantes y sobrantes.
6. Un pool de conexiones propio y sencillo (una cola de objetos `Connection`).
7. Mover el cálculo del cierre de turno a un procedimiento almacenado PL/SQL invocado con `CallableStatement`.

---

13. Rúbrica de evaluación

Criterio	Peso
Modelo de dominio y restricciones de integridad	20 %
Seguridad: login, permisos, reversión con rol superior	25 %
Reglas de negocio de caja (turno, cortes, cuadre)	25 %
Calidad de pantallas y manejo de errores	10 %
Pruebas unitarias	15 %
Código limpio y separación de capas	5 %

14. Entregables

- Repositorio Git con commits por fase (`fase-1-modelo`, `fase-2-login`, etc.).
- `README` con instrucciones de ejecución y usuarios de prueba.
- Diagrama entidad-relación del modelo final y scripts SQL de Oracle XE (carpeta sql/).
- Capturas de las 12 pantallas y resultado del escenario de la Fase 11.